Lab 2 Report

File Upload and Command Execution Security Assessment

Student Name: Daniyal Ahmed

Registration Number: C11/26/EHIT/17331

Cohort: Ethical Hacking Internship Training

Course: Offensive Security Operations I (CIP-A104)

Submission Date: 29 August 2026

Assessment: Lab 2

1. Scope and Environment

This assessment evaluated file upload validation and OS command execution trust boundaries in two deliberately vulnerable applications: DVWA and OWASP Mutillidae II (v2.12.7), both running on dedicated local nginx instances (ports 42001 and 42002) inside an isolated Kali VM. Testing used only learner-created harmless marker files and non-destructive commands (`whoami`), in line with the lab's strict constraints. No executable payloads, reverse shells, or destructive commands were used at any point.

Environment availability was confirmed before testing: DVWA returned HTTP 302 (login redirect) and Mutillidae returned HTTP 200 on their respective ports.

2. Evidence Preparation

Two baseline marker files and one content/extension mismatch file were created and hashed prior to testing.

File	SHA-256	Size	Detected Type
marker1.txt	69d456799c6477801aa53979bf821cbb12da565010ae370e85a9e8de3d22e0a6	34 B	ASCII text
marker2_valid.jpg	829961027d4a48a5b2e43245f8e3a88e03113766f26f5921083e11155bcfc75b	23 B	ASCII text
marker3_mismatch.jpg	883d1b5f7be61d3ff596fbf2d3d86d4fa16075fc6e403e12e041cbcb110ac11d	45 B	PHP script, ASCII text



Figure 1: Marker file creation, SHA-256 hashing and type detection.

3. Upload Baseline

3.1 DVWA (Security: Low)

marker1.txt was uploaded via `POST /vulnerabilities/upload/` and was accepted with no restriction, despite the form requesting an image. The application stored the file unmodified at `../../hackable/uploads/marker1.txt`.

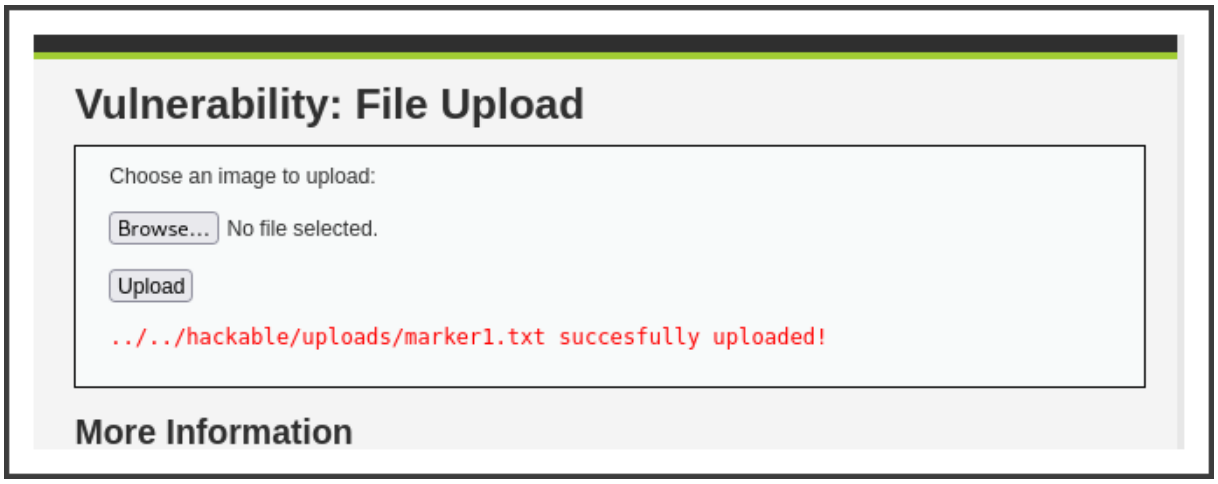


Figure 2: DVWA baseline upload accepted with no type restriction.

3.2 Mutillidae (Security Level 0)

marker2_valid.jpg was uploaded via `POST /index.php?page=upload-file.php`. The application explicitly reported **"Validation not performed"** and stored the file to `/tmp/marker2_valid.jpg`, reporting a MIME type of image/jpeg based purely on the client-supplied Content-Type header.

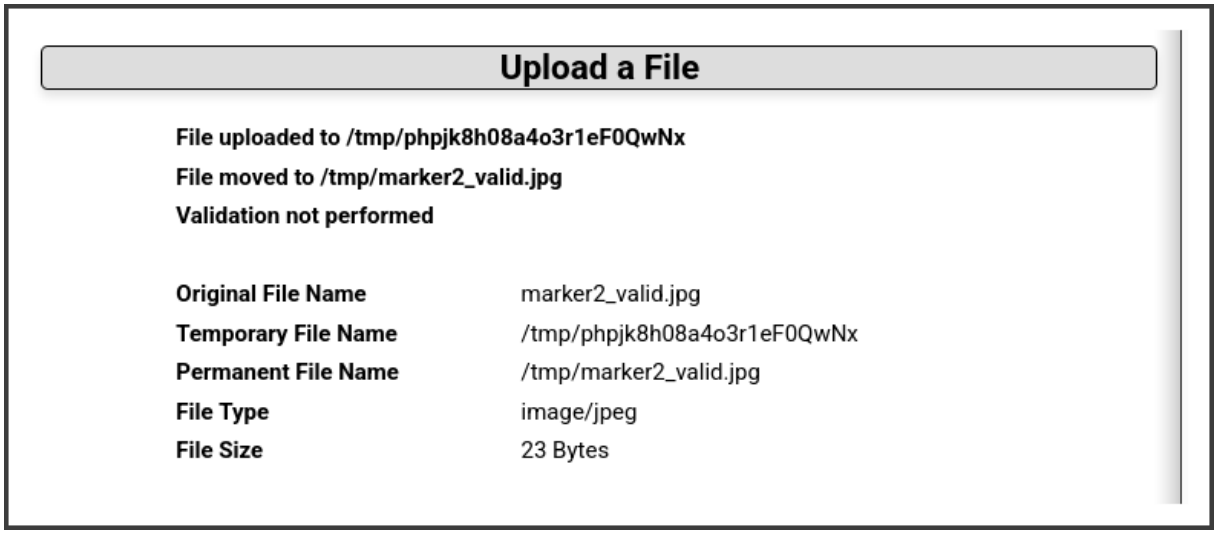


Figure 3: Mutillidae baseline upload, validation explicitly not performed.

4. Validation Comparison — Mismatch Test

marker3_mismatch.jpg contains PHP-tag text content but carries a .jpg extension. The system `file` utility correctly identified it as a "PHP script, ASCII text" rather than an image.

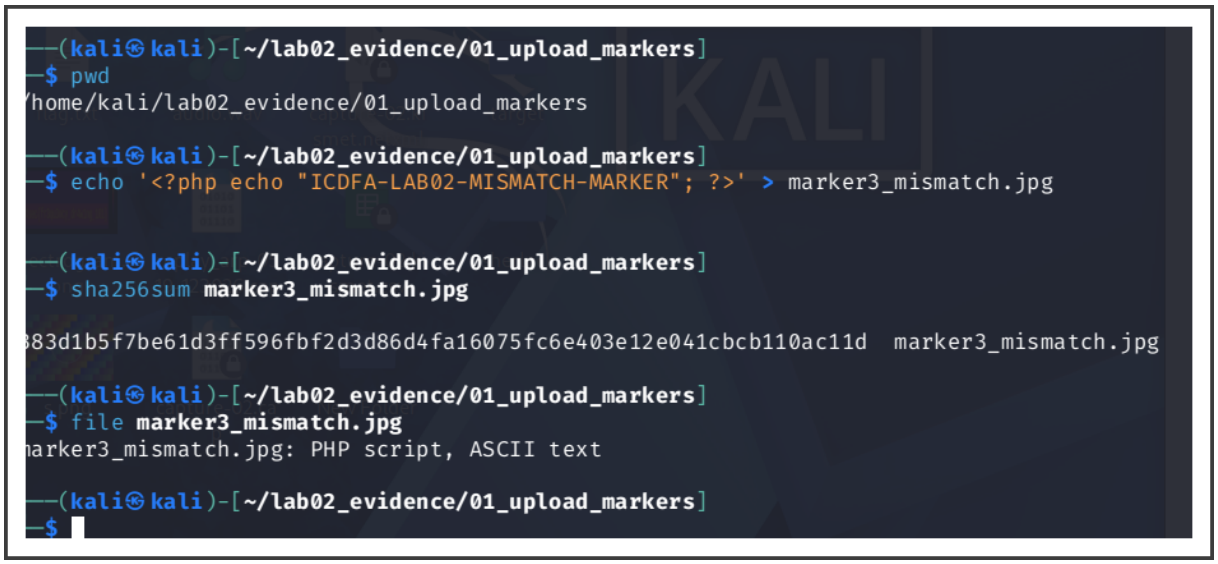


Figure 4: Mismatch marker created; file type correctly identified as a PHP script.

Both applications accepted the mismatched file without any warning:

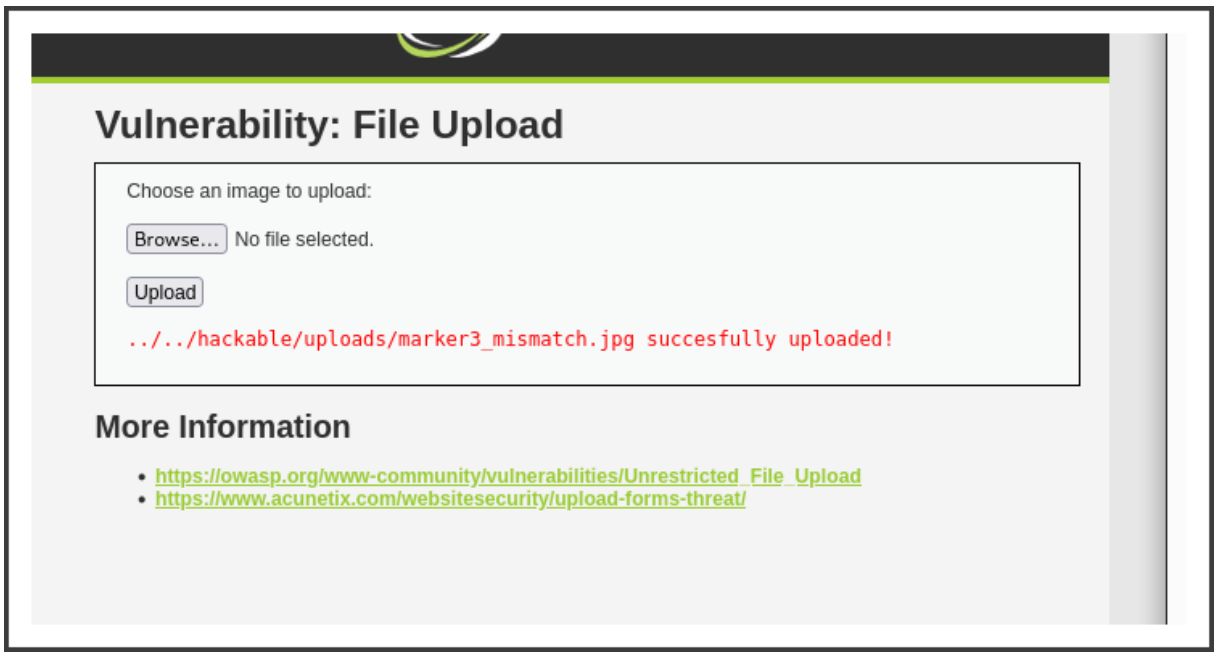


Figure 5: DVWA accepts the mismatched file with no content or magic-byte check.

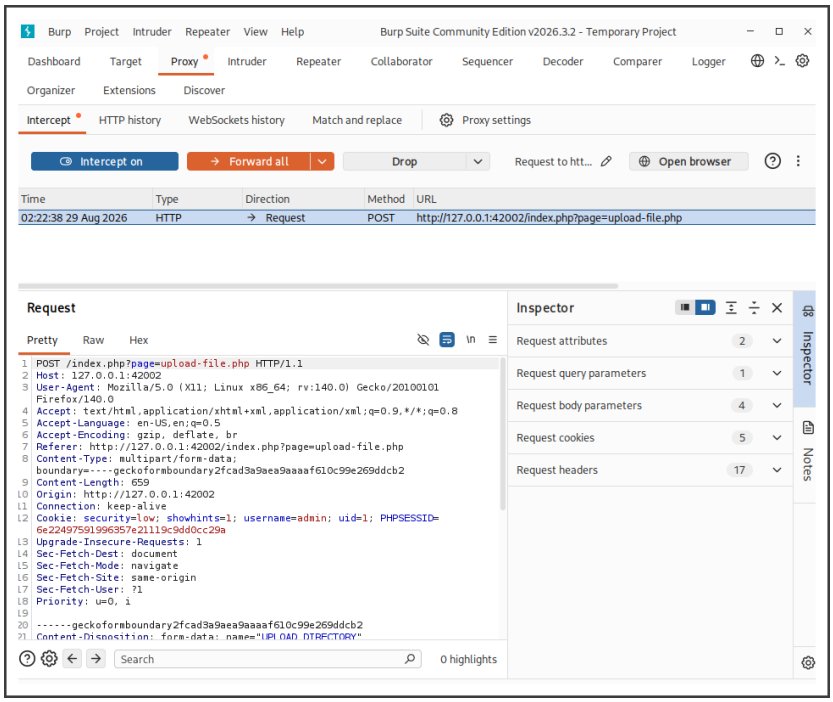


Figure 6: Mutillidae accepts the mismatched file, again reporting no validation performed.

Validation conclusion: Neither application performs server-side content or magic-byte validation at the insecure baseline. Both rely solely on the client-supplied filename extension and/or Content-Type header — both of which are fully attacker-controlled fields in the multipart request. This confirms that **browser-only (or extension/MIME-only) validation is insufficient**, since it inspects data the client itself constructs and can therefore be trivially bypassed by crafting the request directly (e.g. via an intercepting proxy) rather than through the browser's file picker.

5. Storage Assessment

Direct HTTP retrieval was attempted against each application's upload storage location to determine whether uploaded content sits inside a web-accessible, and therefore potentially executable, path.

Request	Result
GET /hackable/uploads/marker3_mismatch.jpg (DVWA)	HTTP 200 – directly retrievable
GET /marker3_mismatch.jpg (Mutillidae)	HTTP 404 – not found
GET /tmp/marker3_mismatch.jpg (Mutillidae)	HTTP 404 – not web-accessible



Figure 7: DVWA upload path returns HTTP 200; Mutillidae's /tmp/ storage returns HTTP 404.

Storage conclusion: DVWA stores uploads inside its web root with no renaming and no execution-path segregation — any accepted file is immediately and directly retrievable over HTTP. Mutillidae stores uploads to `/tmp/`, which is outside the web-served directory tree and not reachable via HTTP by default. Both applications share the same root cause (no content validation), but the practical blast radius differs significantly due to storage architecture: DVWA's design converts a missing validation check directly into a retrievable/executable artefact, whereas Mutillidae's would require a secondary vulnerability (e.g. local file inclusion) to reach the stored file.

6. Command Injection Baseline

6.1 DVWA — Ping a Device (Security: Low)

Submitting a normal IP (127.0.0.1) to `POST /vulnerabilities/exec/` (parameter `ip`) returned standard ping output, confirming the backend directly invokes a system ping command using the supplied value.

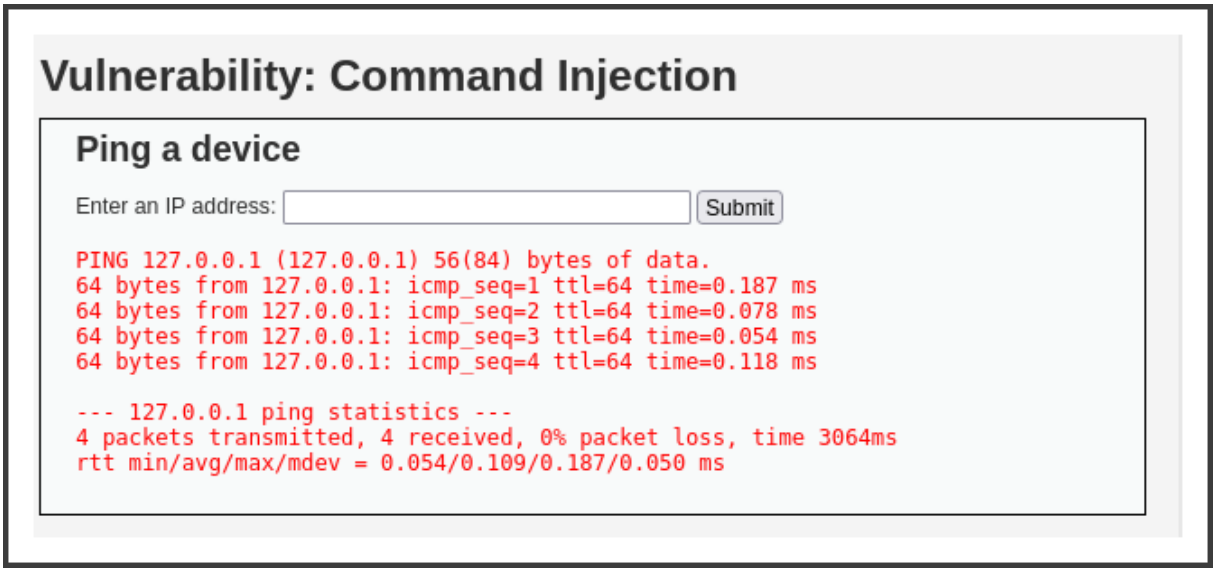


Figure 8: DVWA command injection baseline – normal ping output for 127.0.0.1.

6.2 Mutillidae — DNS Lookup (Security Level 0)

Submitting 127.0.0.1 to `POST /index.php?page=dns-lookup.php` (parameter `target_host`) returned a normal reverse DNS result, confirming the backend runs a lookup command against the supplied host.

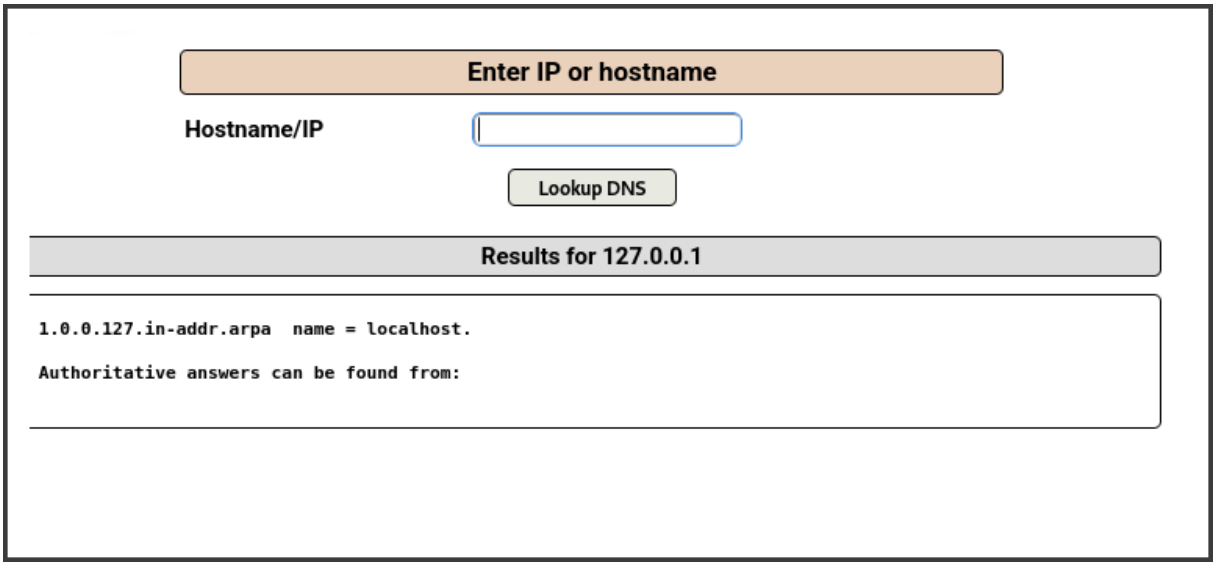


Figure 9: Mutillidae DNS Lookup baseline – normal reverse-DNS result for 127.0.0.1.

7. Minimal Execution Proof

A harmless separator (&&) plus the non-destructive command `whoami` was appended to the normal input on both applications to determine whether user input is treated as command structure rather than plain data.

7.1 DVWA

Input: `127.0.0.1 && whoami`. Result: normal ping output followed immediately by _**dvwa**, the system account under which the application executes.

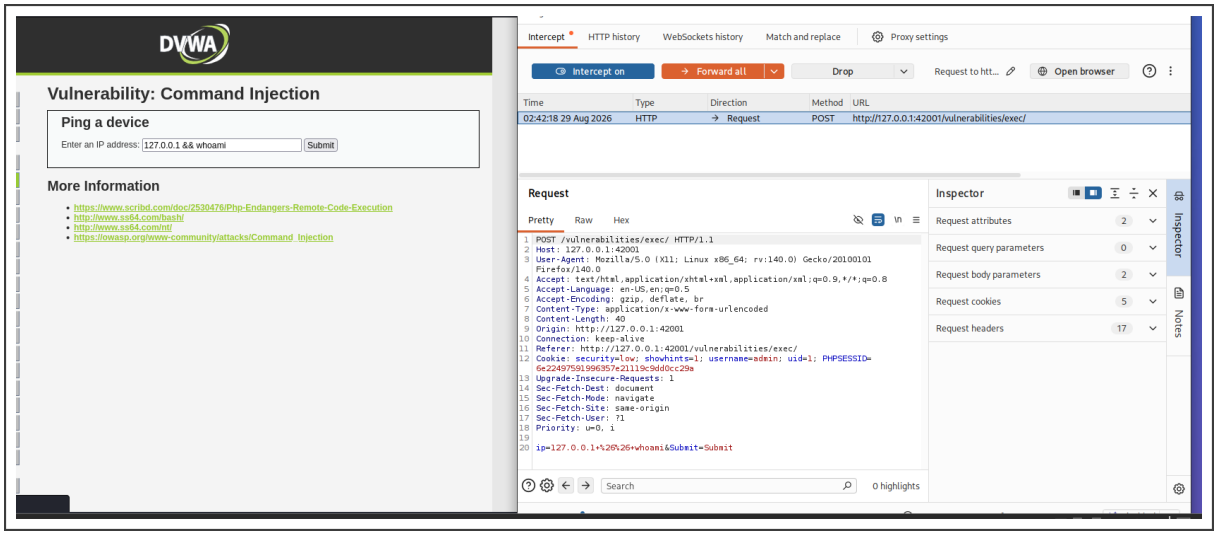


Figure 10: DVWA – whoami output (_dvwa) appended after ping, with the raw request showing the unsanitised && operator.

7.2 Mutillidae

Input: `127.0.0.1 && whoami`. Result: normal DNS output followed by **_mutillidae**, a distinct low-privilege system account consistent with the dedicated service user created for Mutillidae in Lab 1.

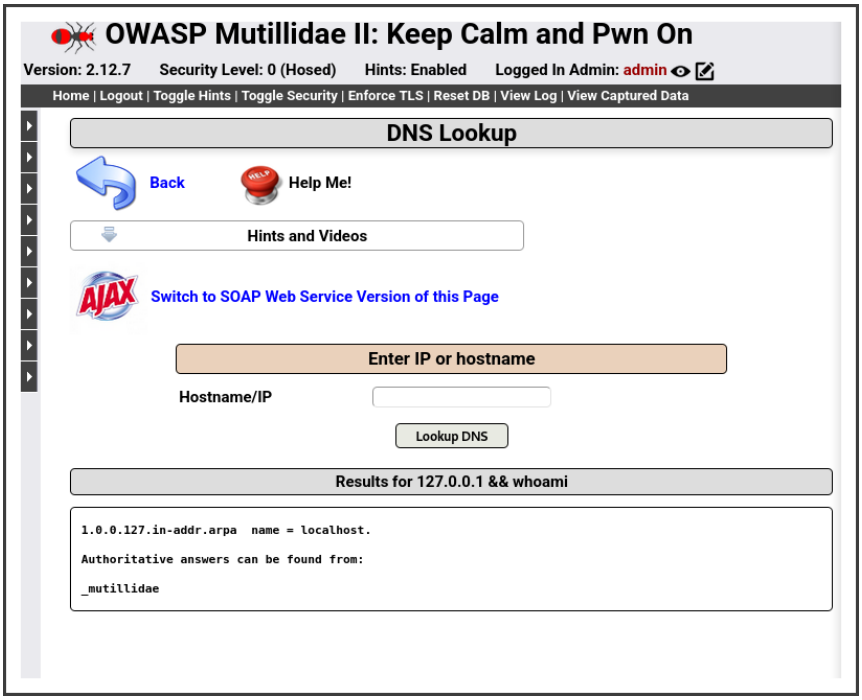


Figure 11: Mutillidae – whoami output (_mutillidae) appended after the DNS result.

Proof rationale: Appending a shell metacharacter followed by an unrelated command (whoami) and observing that command's own output appear in the response is sufficient to prove the input was parsed as executable command structure, not as inert data passed to the intended program. This is a minimal, non-destructive test: it requires no destructive payload, no persistence and no network callback, yet unambiguously demonstrates the trust boundary failure and reveals the exact execution context (the running system account) without touching any unrelated file, credential or system.

8. Impact Assessment

Application	Vulnerability	Observed Impact	Potential Impact (not tested)
DVWA	Unrestricted file upload	File stored in web root, directly retrievable via HTTP	A server-executable extension would allow remote code execution as _dvwa
DVWA	OS command injection (ip parameter)	whoami executed, returned _dvwa	Arbitrary command execution limited to _dvwa's privileges
Mutillidae	Unrestricted file upload	File stored to /tmp/, not web-accessible	Impact contained unless chained with LFI/RFI reaching /tmp/
Mutillidae	OS command injection (target_host parameter)	whoami executed, returned _mutillidae	Arbitrary command execution limited to _mutillidae's privileges

9. Secure Mode Comparison and Retest

9.1 DVWA — Security: High

Upload: marker3_mismatch.jpg was rejected outright: "Your image was not uploaded. We can only accept JPEG or PNG images." This is a genuine improvement over Low, which performed no check at all.



Figure 12: DVWA High – mismatched file rejected by type validation.

Command injection: The payload `127.0.0.1 && whoami` and two further separator variants (`|` and `;`, no leading space) were each submitted directly via an intercepting proxy at Security: High. All three produced no output and no whoami result, confirming the common shell metacharacters are filtered or stripped server-side before the ping command is built.

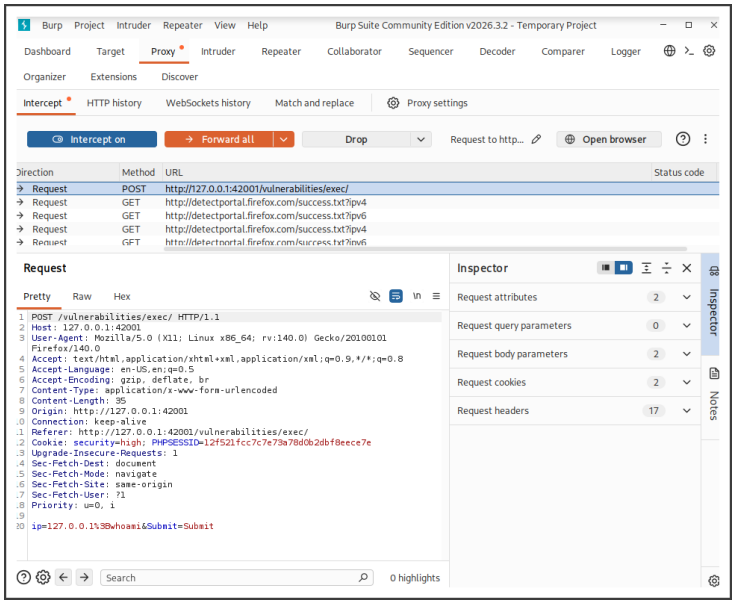


Figure 13: DVWA High – request captured with security=high and a semicolon payload; no command output was returned for any of the three separators tested.

9.2 Mutillidae — Security Level 5 (Secure)

Command injection: Submitting the same payload through the browser triggered a client-side JavaScript alert ("Malicious characters are not allowed") and the request was never sent.



Figure 14: Mutillidae Level 5 – client-side JavaScript blocks the payload before submission.

The same payload was then sent directly via an intercepting proxy, bypassing the browser entirely, to determine whether the protection is enforced server-side or only in JavaScript.

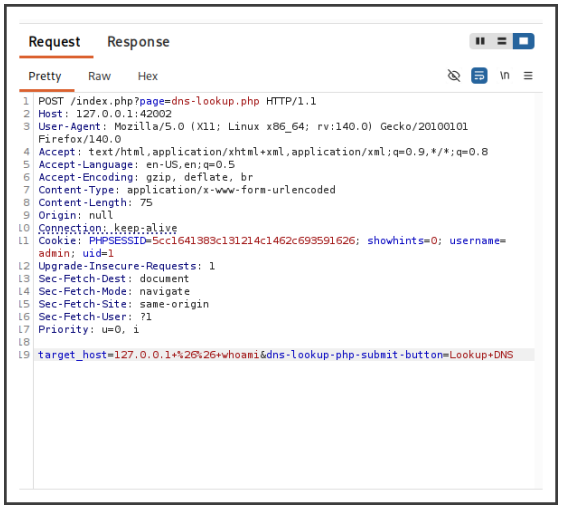


Figure 15: Crafted request sent directly to the server, bypassing the browser's JavaScript check.

The server response returned an "Invalid Input" error with no command output, and the page source disclosed the exact filter pattern in use: `/[;&|<>]/`. This confirms Mutillidae Level 5 enforces a genuine server-side character blacklist in addition to its client-side JavaScript check — a real, non-cosmetic improvement over Level 0, not merely client-side theatre.

Upload: marker3_mismatch.jpg was still accepted at Level 5, though the application now reported "Validation performed" (versus "Validation not performed" at Level 0).



Figure 16: Mutillidae Level 5 – upload accepted; message changes to "Validation performed" but the content/extension mismatch is still not caught.

This indicates that whatever check now runs at Level 5 validates extension and/or client-supplied MIME type, but not actual file content or magic bytes — a partial, incomplete improvement.

9.3 Which control produced the greatest improvement?

DVWA's command injection control at High produced the most complete improvement observed: all three tested separators were fully neutralised server-side with no bypass found within the tested scope. Mutillidae's Level 5 command injection control was a close second, being genuinely server-side enforced, but its filter is a character blacklist (documented in the client-side script and consistent with the server behaviour), which is inherently more brittle than an allow-list approach. Both applications' upload controls improved the least: DVWA introduced extension checking but was not retested against magic-byte spoofing, and Mutillidae's "validation performed" message did not, in practice, catch the same content mismatch that a magic-byte check would.

10. Remediation Recommendations

Weakness	Root Cause	Recommended Control
Unrestricted file upload (both apps)	Validation relies on client-supplied filename/extension or MIME type only	Server-side allow-list of extensions; verify actual file content via magic-byte/signature check; re-encode images server-side; generate a random stored filename
Web-accessible upload storage (DVWA)	Uploads stored inside the web root with no execution-path segregation	Store uploads outside the web root, or in a location with execute permissions disabled and served only through a controlled download handler
OS command injection (both apps)	User input concatenated directly into a shell command string	Avoid shell invocation entirely; use language-native library calls (e.g. a DNS resolution library) or parameterised subprocess execution with a strict allow-list of expected input format (e.g. valid IPv4/hostname regex)
Blacklist-based input filtering (DVWA High, Mutillidae Level 5)	Filtering blocks known-bad characters/sequences rather than defining what is explicitly allowed	Replace blacklists with strict allow-list input validation (e.g. anchor-matched IP/hostname pattern) so any unexpected structure is rejected by default
Client-side-only assumptions	JavaScript validation can be bypassed by submitting requests directly to the server	Treat client-side checks purely as a usability aid; always enforce equivalent or stricter checks server-side

11. Cleanup and Restoration

All learner-created marker files were removed from both applications' upload storage locations, and the environment was verified to be free of listeners, temporary accounts or persistent artefacts.



Figure 17: Removal of marker files from DVWA and Mutillidae storage locations.



Figure 18: Verification – only the expected nginx listeners remain on 42001/42002; no reverse-shell or unexpected processes found.

12. Analysis Questions — Summary

Why is browser-only upload validation insufficient?

Client-side and extension/MIME-based checks inspect data the client itself supplies and controls. An attacker can bypass the browser entirely and submit a crafted multipart request with any filename or Content-Type value directly to the server, as demonstrated in Section 4.

What proves that command input was treated as executable structure rather than data?

Appending a shell metacharacter and an unrelated command (whoami) and observing that command's own distinct output (the system account name) in the response proves the input was parsed as command syntax, not as an inert argument to the intended ping/lookup program (Section 7).

Why is minimal proof sufficient for both weaknesses?

A single non-destructive marker file or a single harmless command (whoami) unambiguously demonstrates the trust boundary failure and reveals the execution context without requiring destructive payloads, persistence, or access to unrelated systems — satisfying the assessment objective while remaining within the lab's strict non-destructive constraints.

Which control produced the greatest improvement in stronger mode?

DVWA's Security: High command injection filtering, which fully blocked all three tested shell metacharacter variants server-side (Section 9.3).